

Data Analysis with Python 2024–2025

Parte 3: NumPy (Parte 1)

Caderno de Exercícios

Tradução e Adaptação: Universidade de Helsinque (MOOC.fi)

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**.

Conteúdo

1 Exercícios - Capítulo 2 (NumPy)

Exercício 2.11 – Linhas e colunas

Sumário

- **Objetivo:** Escrever as funções `get_rows` e `get_columns` que recebam uma matriz bidimensional e retornem, respectivamente, listas contendo as linhas e colunas formatadas como vetores unidimensionais.
- **Dica:** A operação de transposição (método `.T`) pode facilitar a extração das colunas ao ser combinada com a função de linhas.

Exercício 2.12 – Vetores-linha e vetores-coluna

Sumário

- **Objetivo:** Extrair linhas e colunas de uma matriz 2D de formato (n, m) , mas mantendo estritamente a estrutura 2D original dos vetores.
- **Formato Esperado:** A função `get_row_vectors` deve retornar vetores no formato $(1, m)$. A função `get_column_vectors` deve retornar vetores no formato $(n, 1)$. Utilize *slicing* inteligente que preserva a dimensão.

Exercício 2.13 – Diamante

Sumário

- **Objetivo:** Criar a função `diamond(n)` que gera uma matriz bidimensional representando a figura de um losango/diamante delimitado por valores 1, enquanto os outros valores permanecem 0.
- **Ferramentas:** Utilize a manipulação de blocos de matrizes unindo `np.eye` (para a geração de diagonais), operações de reflexão (`np.fliplr`, `np.flipud`), e junção com `np.concatenate`.

Exercício 2.14 – Comprimento de vetores

Sumário

- **Objetivo:** Criar a função `vector_lengths(a)` que receba uma matriz de formato (n, m) e calcule o comprimento/norma euclidiana de cada uma das n linhas.
- **Restrições:** É estritamente **proibido o uso de laços** (`for/while`) e não é permitido invocar `scipy.linalg.norm`.
- **Ferramentas Matemáticas:** A solução deve ser vetorizada, baseando-se no fato que a norma Euclidiana é a raiz da soma dos quadrados. Utilize pelo menos as funções `np.sum` e `np.sqrt`.

Exercício 2.15 – Ângulos entre vetores

Sumário

- **Objetivo:** Desenvolver `vector_angles(X, Y)` para calcular simultaneamente os ângulos (em graus) entre vetores representados pelas linhas das matrizes `X` e `Y`.
- **Fórmula:** O cosseno do ângulo é obtido dividindo o produto interno escalar dos vetores pelo produto numérico das suas respectivas normas.
- **Restrições:** O uso de laços é proibido. Para contornar imperfeições e imprecisões no processamento em float, antes de passar o cosseno para o `np.arccos`, garanta que ele esteja preso no limite entre `[-1.0, 1.0]` utilizando o método `np.clip`. Aplique `np.degrees` na saída.

Exercício 2.16 – Tabuada revisitada

Sumário

- **Objetivo:** Construir a função `multiplication_table(n)` que retorne um array bidimensional de tamanho `(n, n)`, em que cada célula `i, j` possui o valor exato da multiplicação de seus eixos `(i * j)`.
- **Restrições:** Não utilizar laços `for` na solução final.
- **Ferramentas:** Sua resolução obrigatoriamente exigirá a aplicação do conceito de *broadcasting*. Instancie os multiplicadores base via `np.arange`, altere seus eixos utilizando `reshape`, e delegue a carga matemática aos operadores universais do NumPy.